notebook2

The purpose of this notebook is to record my decision making during the model building process. I will start with a simple model and work up to the final model, that way I can check computation and model fit along the way.

Load in packages.

```
library(cmdstanr)
library(dplyr)
library(tidyr)
library(ggplot2)
library(bayesplot)
library(posterior)
library(here)
```

Overall relative growth rate and initial size

First, the simplest model is just fitting a single intercept and slope to the entire data set. The plants are growing exponentially during this life-stage: $s_t = s_0 e^{rt}$. Here s_t is size at time t, and s_0 is the original size. r is the growth rate, here relative growth rate, and t is the time since germination of the seedling. By taking the log of this we get: $\log(s_t) = \log(s_0) + rt$ which we can fit using a normal linear regression.

Here's the model:

$$
\begin{aligned}
\log(s) &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \log(s_0) + rgr \times age \\
\log(s_0) &\sim \text{Normal}(0, 1) \\
rgr &\sim \text{Normal}(0.24, 0.1) \\
\sigma &\sim \text{Exponential}(1)
\end{aligned}
\tag{1}
$$

Stan models are broken into three main blocks. Data describes the data, here we are telling Stan that there are N data points, there is a vector of size N l_size, that will be the log sizes and a vector of length N of ages. The parameters block lays out the unknown parameters from the model. Here l_size0 is the size at time 0. rgr is the relative growth rate, and sigma is the standard deviation of the residuals. This is a bog standard linear regression. Comments in Stan follow a double forward slash: `//`. The prior distribution of *rgr* comes from a previous study.

Here is the Stan model for model 1.

```
data{
  int N; // number of observations
  vector[N] l_size; // size observations
  vector[N] age; // age of observations
}
parameters{
  real l_size0; // log size at time 0
  real rgr; // relative growth rate
  real<lower=0> sigma; // standard deviation of errors
}
model{
  // priors
  l_size0 ~ normal(0, 1);
  rgr ~ normal(0.24, 0.1);
  sigma ~ exponential(1);

  // likelihood
  l_size ~ normal(l_size0 + rgr*age, sigma);
}
```

Now fit the model to the data:

```
# read in the seedling size data
sizes <- read.csv(here("data/wrangled/seedling_areas.csv"))
sizes <- sizes %>%
  # remove the germination day and individual lc_2_2_20, doesn't have
  # seed data, so I'm removing it
  filter(age > 0,
         paste0(tray, "_", block, "_", ind) != "lc_2_2_20")
```

```
# make a data set for Stan
dat <- list(
```

```r
  N = nrow(sizes),
  l_size = log(sizes$area),
  age = sizes$age
)

# compile the model
mod <- cmdstan_model(here("notebooks/model.stan"))

# fit the model
fit <- mod$sample(
  data = dat,
  chains = 4,
  parallel_chains = 4,
  show_messages = F
)
```

```
Chain 1 Informational Message: The current Metropolis proposal is about to be rejected becaus

Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/tmp/RtmpvF
8e4451924fa.stan', line 18, column 2 to column 44)

Chain 1 If this warning occurs sporadically, such as for highly constrained variable types li

Chain 1 but if this warning occurs often then your model may be either severely ill-
conditioned or misspecified.

Chain 1

Chain 1 Informational Message: The current Metropolis proposal is about to be rejected becaus

Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/tmp/RtmpvF
8e4451924fa.stan', line 18, column 2 to column 44)

Chain 1 If this warning occurs sporadically, such as for highly constrained variable types li

Chain 1 but if this warning occurs often then your model may be either severely ill-
conditioned or misspecified.

Chain 1
```

```
Chain 1 Informational Message: The current Metropolis proposal is about to be rejected becaus

Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/tmp/RtmpvF
8e4451924fa.stan', line 18, column 2 to column 44)

Chain 1 If this warning occurs sporadically, such as for highly constrained variable types li

Chain 1 but if this warning occurs often then your model may be either severely ill-
conditioned or misspecified.

Chain 1

Chain 1 Informational Message: The current Metropolis proposal is about to be rejected becaus

Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/tmp/RtmpvF
8e4451924fa.stan', line 18, column 2 to column 44)

Chain 1 If this warning occurs sporadically, such as for highly constrained variable types li

Chain 1 but if this warning occurs often then your model may be either severely ill-
conditioned or misspecified.

Chain 1

Chain 3 Informational Message: The current Metropolis proposal is about to be rejected becaus

Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/tmp/RtmpvF
8e4451924fa.stan', line 18, column 2 to column 44)

Chain 3 If this warning occurs sporadically, such as for highly constrained variable types li

Chain 3 but if this warning occurs often then your model may be either severely ill-
conditioned or misspecified.

Chain 3

Chain 3 Informational Message: The current Metropolis proposal is about to be rejected becaus
```

```
Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/tmp/RtmpvF
8e4451924fa.stan', line 18, column 2 to column 44)

Chain 3 If this warning occurs sporadically, such as for highly constrained variable types li

Chain 3 but if this warning occurs often then your model may be either severely ill-
conditioned or misspecified.

Chain 3

Chain 4 Informational Message: The current Metropolis proposal is about to be rejected becaus

Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/tmp/RtmpvF
8e4451924fa.stan', line 18, column 2 to column 44)

Chain 4 If this warning occurs sporadically, such as for highly constrained variable types li

Chain 4 but if this warning occurs often then your model may be either severely ill-
conditioned or misspecified.

Chain 4

Chain 4 Informational Message: The current Metropolis proposal is about to be rejected becaus

Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/tmp/RtmpvF
8e4451924fa.stan', line 18, column 2 to column 44)

Chain 4 If this warning occurs sporadically, such as for highly constrained variable types li

Chain 4 but if this warning occurs often then your model may be either severely ill-
conditioned or misspecified.

Chain 4
```

```
fit$summary()
```

```
# A tibble: 4 x 10
  variable     mean   median      sd     mad       q5      q95  rhat ess_bulk
  <chr>       <dbl>    <dbl>   <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
1 lp__     1106.    1106.    1.21    0.982   1103.    1107.     1.00    1425.
2 l_size0     0.438    0.438 0.0147  0.0147     0.415    0.463  1.00    1713.
3 rgr         0.231    0.231 0.00154 0.00151    0.228    0.233  1.00    1901.
4 sigma       0.487    0.487 0.00471 0.00474    0.479    0.495  1.00    1731.
# i 1 more variable: ess_tail <dbl>
```

```
fit$diagnostic_summary()
```

```
$num_divergent
[1] 0 0 0 0

$num_max_treedepth
[1] 0 0 0 0

$ebfmi
[1] 0.9186144 1.0783213 1.1891593 1.0477622
```

The model fits well, there are no divergences, everything looks good. Looks like the mean relative growth rate is about 0.23 with mean log size0 of about 0.47 and a residual standard error of about 0.51. Good benchmarks, I'm not too interested in these values though, next is to start inferring individual growth rates.

Individual growth rates

The next step in the model is to start fitting individual growth rates, as all the individuals were measured at multiple time points, we can estimate individual growth rate. The model is similar to model 1 except now we allow relative growth rate to vary with individual. First is that rgr from model 1 now becomes the mean relative growth rate: μ_{rgr}. Now we need to fit a standard deviation for the distribution of relative growth rates: σ_{rgr}. With these additions the model of varying slopes becomes:

$$
\begin{aligned}
\log(s) &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \log(s_0) + rgr \times age \\
\log(s_0) &\sim \text{Normal}(0, 1) \\
\sigma &\sim \text{Exponential}(1) \\
rgr &\sim \text{Normal}(\mu_{rgr}, \sigma_{rgr}) \\
\mu_{rgr} &\sim \text{Normal}(0.24, 0.1) \\
\sigma_{rgr} &\sim \text{Exponential}(1)
\end{aligned}
\tag{2}
$$

The Stan program for this model is:

```
data{
  int N; // number of observations
  int J; // number of individuals
  vector[N] l_size; // size observations
  vector[N] age; // age of observations
  array[N] int id; // index of individuals
}
parameters{
  real l_size0; // log size at time 0
  real<lower=0> sigma; // standard deviation of errors
  real mu_rgr; // mean relative growth rate
  real<lower=0> sigma_rgr;
  vector[J] rgr;
}
model{
  // priors
  l_size0 ~ normal(0, 1);
  sigma ~ exponential(1);
  mu_rgr ~ normal(0.24, 0.1);
  sigma_rgr ~ exponential(1);
  rgr ~ normal(mu_rgr, sigma_rgr);

  // likelihood
  // Define the expected value for each observation
  vector[N] mu;
  for(i in 1:N){
    mu[i] = l_size0 + rgr[id[i]] * age[i];
  }
  l_size ~ normal(mu, sigma);
}
```

```r
# get the data ready
d <- sizes %>%
  mutate(id = paste0(tray, "_", block, "_", ind),
         id = as.integer(factor(id)))
head(d)
```

```
  tray block ind   area age cross mother father id
1 lc_1     1   1  0.989   1    f2  B12_1 B12_11  1
2 lc_1     1   1  4.504   4    f2  B12_1 B12_11  1
3 lc_1     1   1  6.807   6    f2  B12_1 B12_11  1
4 lc_1     1   1  8.934   8    f2  B12_1 B12_11  1
5 lc_1     1   1 14.791  11    f2  B12_1 B12_11  1
6 lc_1     1   1 18.324  12    f2  B12_1 B12_11  1
```

```r
# compile the model and get the data ready
mod <- cmdstan_model(here("notebooks/model.stan"))

dat <- list(
  N = nrow(d),
  J = length(unique(d$id)),
  l_size = log(d$area),
  age = d$age,
  id = d$id
)

fit <- mod$sample(
  data = dat,
  chains = 4,
  parallel_chains = 4,
  show_messages = F,
  show_exceptions = F
)

fit$diagnostic_summary()
```

```
$num_divergent
[1] 0 0 0 0

$num_max_treedepth
[1] 0 0 0 0
```

```
$ebfmi
[1] 0.8181997 0.9053322 0.9535700 0.9456651
```

```
fit$summary()
```

```
# A tibble: 889 x 10
   variable       mean    median       sd     mad      q5     q95  rhat ess_bulk
   <chr>         <dbl>     <dbl>    <dbl>   <dbl>   <dbl>   <dbl> <dbl>    <dbl>
 1 lp__       6611.     6611.     23.1     2.31e+1 6.57e+3 6.65e+3 1.01     1198.
 2 l_size0       0.471     0.471   0.00806 7.97e-3 4.58e-1 4.84e-1 1.00     1036.
 3 sigma         0.251     0.251   0.00273 2.75e-3 2.47e-1 2.56e-1 1.00     3922.
 4 mu_rgr        0.223     0.223   0.00200 1.96e-3 2.20e-1 2.26e-1 1.00     2257.
 5 sigma_rgr     0.0511    0.0511  0.00145 1.44e-3 4.88e-2 5.35e-2 1.00     4090.
 6 rgr[1]        0.210     0.210   0.0126  1.30e-2 1.89e-1 2.30e-1 1.000    6758.
 7 rgr[2]        0.248     0.247   0.0122  1.23e-2 2.28e-1 2.68e-1 1.00     6144.
 8 rgr[3]        0.287     0.287   0.0236  2.42e-2 2.48e-1 3.26e-1 1.00     6504.
 9 rgr[4]        0.217     0.217   0.0104  1.02e-2 2.00e-1 2.35e-1 1.00     5457.
10 rgr[5]        0.215     0.215   0.00866 8.80e-3 2.02e-1 2.30e-1 1.00     7412.
# i 879 more rows
# i 1 more variable: ess_tail <dbl>
```

Everything looks good computationally, Allowing the slopes to vary has eaten up some of the residual variance as expected, but I don't want to really look at them yet, I need to let the intercepts vary first.

Individual growth rates and individual initial sizes

In model 2, I let the slopes vary by individual, but gave all the individuals the same starting size. I'm now going to let the individuals have the different log(size0) as well. Typically in these varying intercepts and varying slopes models, we also try to fit a correlation to the slopes and intercepts. I will work up to that. It is common that introducing covariance matrices makes the computation more difficult, so I'm going to fit the slopes and intercepts "independently" first that way I know whether it was adding the intercept terms or the covariance that messes up the computation later.

Adding a varying log(size0) is similar to varying *rgr* in model 2. We now have a grand mean for log(size0) as well as a variance (standard deviation) term. The model now becomes:

$$
\begin{aligned}
\log(s) &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \log(s_0) + rgr \times age \\
\sigma &\sim \text{Exponential}(1) \\
rgr &\sim \text{Normal}(\mu_{rgr}, \sigma_{rgr}) \\
\mu_{rgr} &\sim \text{Normal}(0.24, 0.1) \\
\sigma_{rgr} &\sim \text{Exponential}(1) \\
\log(s_0) &\sim \text{Normal}(\mu_{s0}, \sigma_{s0}) \\
\mu_{s0} &\sim \text{Normal}(0, 1) \\
\sigma_{s0} &\sim \text{Exponential}(1)
\end{aligned}
\tag{3}
$$

The Stan program for this model is:

```
data{
  int N; // number of observations
  int J; // number of individuals
  vector[N] l_size; // size observations
  vector[N] age; // age of observations
  array[N] int id; // index of individuals
}
parameters{
  real<lower=0> sigma; // standard deviation of errors
  real mu_rgr; // mean relative growth rate
  real<lower=0> sigma_rgr; // standard deviation of growth rates
  vector[J] rgr; // individual relative growth rates
  real mu_s0; // mean size0
  real<lower=0> sigma_s0; // standard deviation of initial log sizes
  vector[J] l_size0; // individual log size0
}
model{
  // priors
  sigma ~ exponential(1);
  mu_rgr ~ normal(0.24, 0.1);
  sigma_rgr ~ exponential(1);
  rgr ~ normal(mu_rgr, sigma_rgr);
  l_size0 ~ normal(mu_s0, sigma_s0);
  mu_s0 ~ normal(0, 1);
  sigma_s0 ~ exponential(1);

  // likelihood
  // Define the expected value for each observation
```

```
  vector[N] mu;
  for(i in 1:N){
    mu[i] = l_size0[id[i]] + rgr[id[i]] * age[i];
  }
  l_size ~ normal(mu, sigma);
}
```

```
# compile the model and get the data ready
mod <- cmdstan_model(here("notebooks/model.stan"))

dat <- list(
  N = nrow(d),
  J = length(unique(d$id)),
  l_size = log(d$area),
  age = d$age,
  id = d$id
)

fit <- mod$sample(
  data = dat,
  chains = 4,
  parallel_chains = 4,
  show_messages = F,
  show_exceptions = F
)

fit$diagnostic_summary()
```

```
$num_divergent
[1] 0 0 0 0

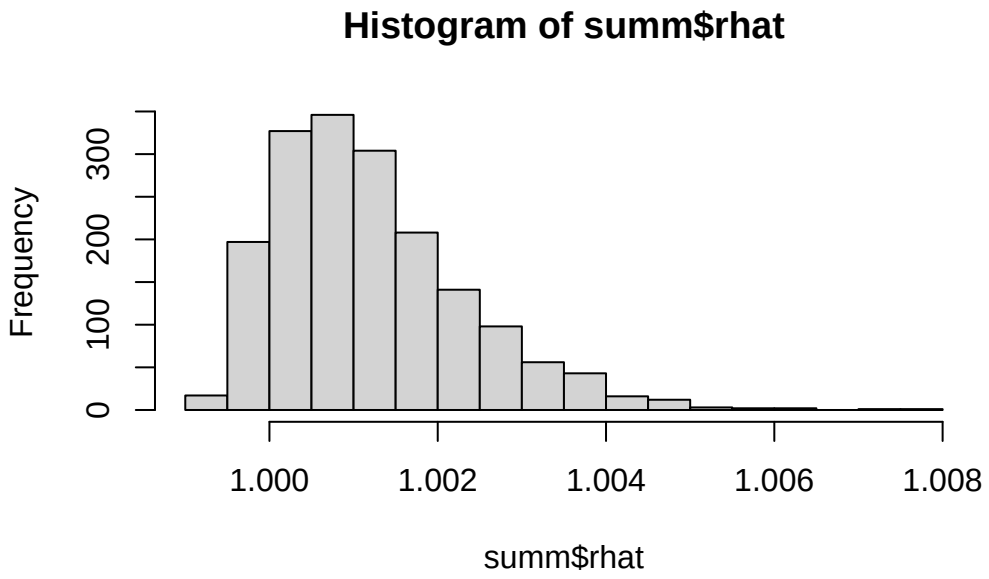
$num_max_treedepth
[1] 0 0 0 0

$ebfmi
[1] 0.7228418 0.8263837 0.8746552 0.8139514
```

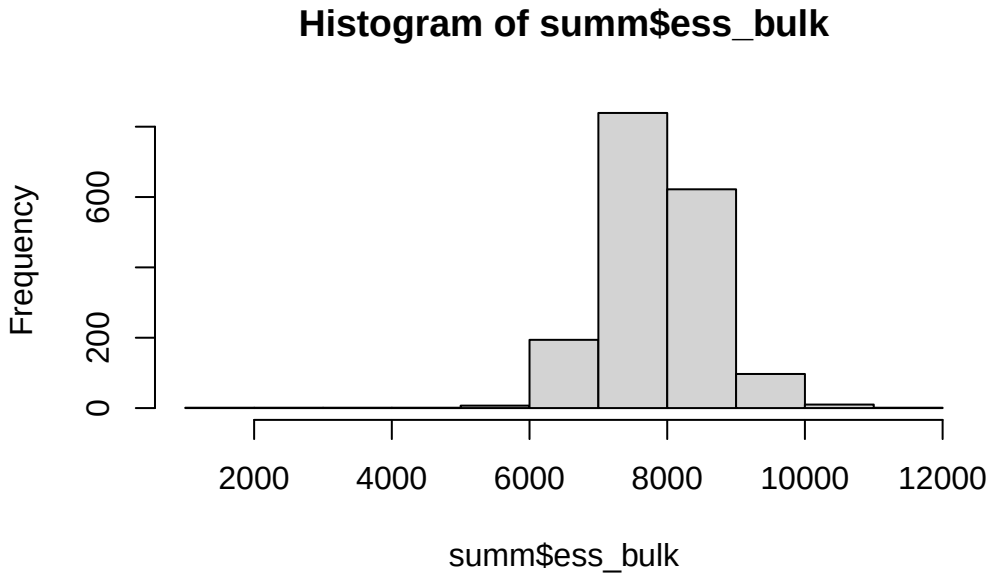
```
summ <- fit$summary()
```

No warnings from the diagnostic summaries. Let's look at the ranges of rhat and the effective sample sizes to just double check.

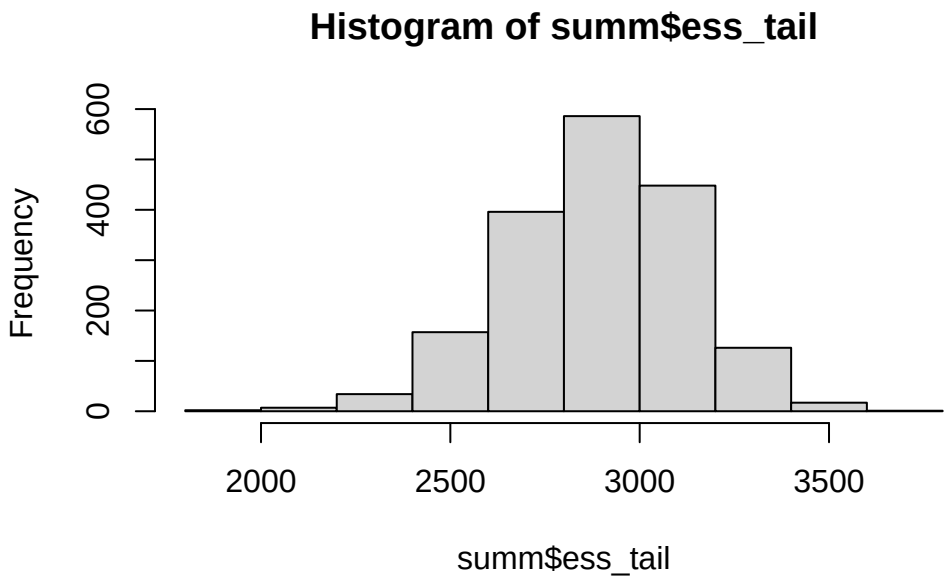
```
hist(summ$rhat)
```




```
hist(summ$ess_bulk)
```



```
hist(summ$ess_tail)
```



Everything looks really good here. Let's let them covary.

Covarying slopes and intercepts

The next step is to try to infer a covariance between the slopes and intercepts. Probably the most straightforward way to do this, since it is easier to interpret standard deviations and correlations than variances and covariances, is to estimate a vector of standard deviations and a correlation matrix:

$$\begin{bmatrix} \log(s0) \\ rgr \end{bmatrix} \sim \text{MVNormal}\left(\begin{bmatrix} \mu_{s0} \\ \mu_{rgr} \end{bmatrix}, \begin{bmatrix} \sigma_{s0} & 0 \\ 0 & \sigma_{rgr} \end{bmatrix} \mathbf{R} \begin{bmatrix} \sigma_{s0} & 0 \\ 0 & \sigma_{rgr} \end{bmatrix} \right)$$

Where $\mathbf{R}$ is a correlation matrix. All that is needed is to replace the independent priors on rgr and l_size0 in model 3 with the above and add a prior on $\mathbf{R}$. To make things a little simpler, I will define a vector $\tau = \begin{bmatrix} \sigma_{s0} & \sigma_{rgr} \end{bmatrix}^T$ make the covariance matrix:

$$\begin{bmatrix} \sigma_{s0} & 0 \\ 0 & \sigma_{rgr} \end{bmatrix} \mathbf{R} \begin{bmatrix} \sigma_{s0} & 0 \\ 0 & \sigma_{rgr} \end{bmatrix} = \text{diag}(\tau)\mathbf{R}\text{diag}(\tau)$$

So the model with covarying slopes and intercepts is:

$$
\begin{aligned}
\log(s) &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \log(s_0) + rgr \times age \\
\sigma &\sim \text{Exponential}(1) \\
\begin{bmatrix} \log(s0) \\ rgr \end{bmatrix} &\sim \text{MVNormal}\left(\begin{bmatrix} \mu_{s0} \\ \mu_{rgr} \end{bmatrix}, \text{diag}(\tau)\mathbf{R}\text{diag}(\tau) \right) \\
\mu_{rgr} &\sim \text{Normal}(0.24, 0.1) \\
\mu_{s0} &\sim \text{Normal}(0, 1) \\
\tau &\sim \text{Exponential}(1) \\
\mathbf{R} &\sim \text{LKJcorr}(2)
\end{aligned}
\tag{4}
$$

The stan program for this equation is:

```
data{
  int N; // number of observations
  int J; // number of individuals
  vector[N] l_size; // size observations
  vector[N] age; // age of observations
  array[N] int id; // index of individuals
}
parameters{
  real<lower=0> sigma; // standard deviation of errors
  real mu_rgr; // mean relative growth rate
  real mu_s0; // mean size0
  array[J] vector[2] Mu; // array of vectors carrying l_size0 and rgr
  vector<lower=0>[2] tau; // standard deviation of l_size0 and rgr
  corr_matrix[2] R;
}
model{
  // priors
  sigma ~ exponential(1);
  mu_rgr ~ normal(0.24, 0.1);
  mu_s0 ~ normal(0, 1);
  tau ~ exponential(1);
  R ~ lkj_corr(2);

  Mu ~ multi_normal([mu_s0, mu_rgr]', quad_form_diag(R, tau));

  // likelihood
  // Define the expected value for each observation
```

```
  vector[N] mu;
  for(i in 1:N){
    mu[i] = Mu[id[i], 1] + Mu[id[i], 2] * age[i];
  }
  l_size ~ normal(mu, sigma);
}
```

```
# compile the model and get the data ready
mod <- cmdstan_model(here("notebooks/model.stan"))

dat <- list(
  N = nrow(d),
  J = length(unique(d$id)),
  l_size = log(d$area),
  age = d$age,
  id = d$id
)

fit <- mod$sample(
  data = dat,
  chains = 4,
  parallel_chains = 4,
  show_messages = F,
  show_exceptions = F
)

fit$diagnostic_summary()
```

```
$num_divergent
[1] 0 0 0 0

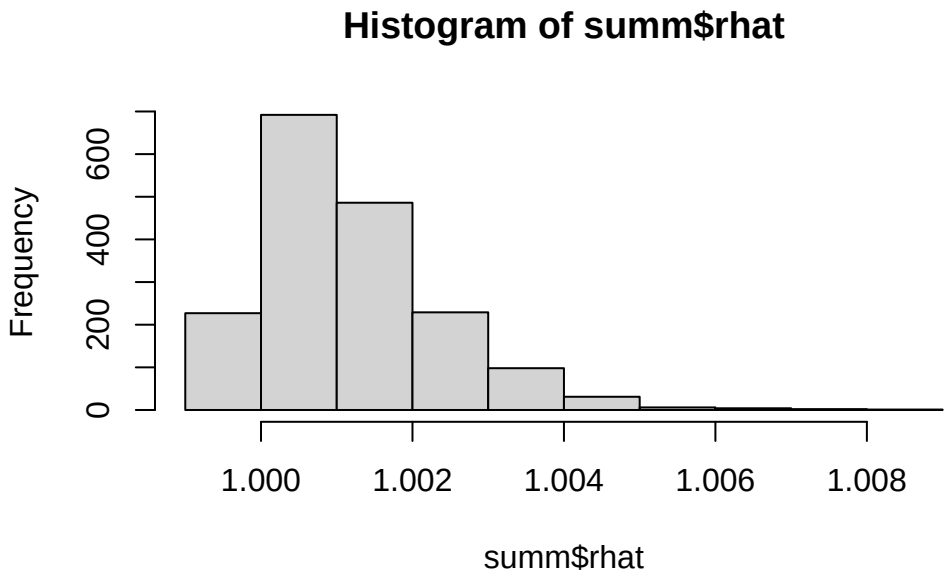
$num_max_treedepth
[1] 0 0 0 0

$ebfmi
[1] 0.7629196 0.7697745 0.7670999 0.7532464
```

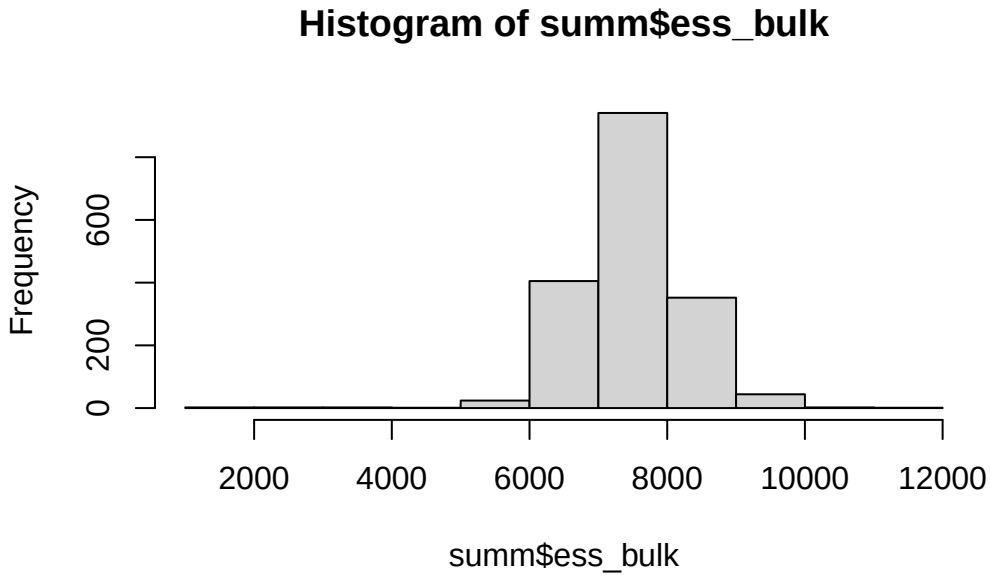
```
summ <- fit$summary()
```

Shockingly, everything looks good. Let's check out rhat and ESS to double check.

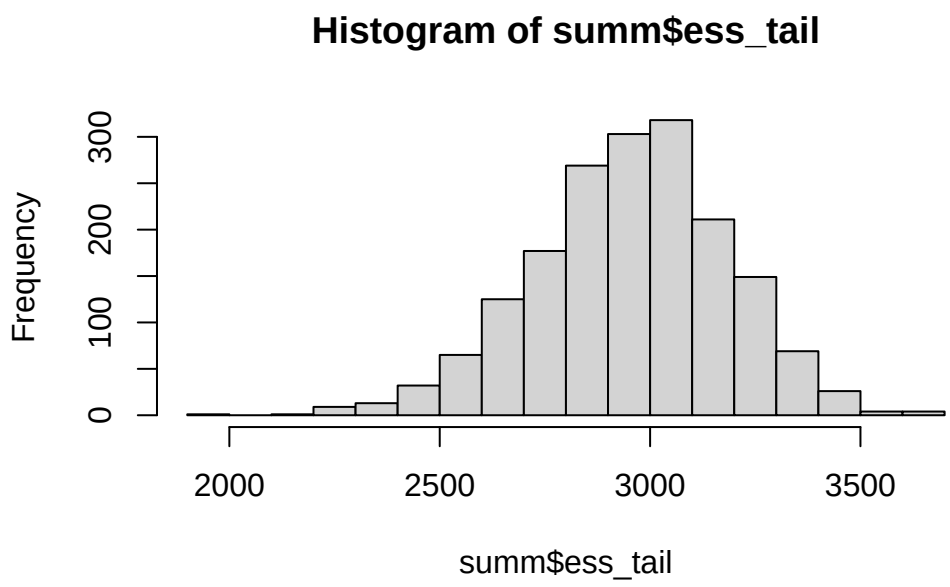
```
hist(summ$rhat)
```




```
hist(summ$ess_bulk)
```



```
hist(summ$ess_tail)
```



Looks great! I think the next step is to add seed size information into the model.

Predicting initial size using seed size

```
seed <- read.csv(here("data/wrangled/seed_size.csv"))
seed <- seed %>%
  select(tray, block, ind, seed_area)
head(seed)
```

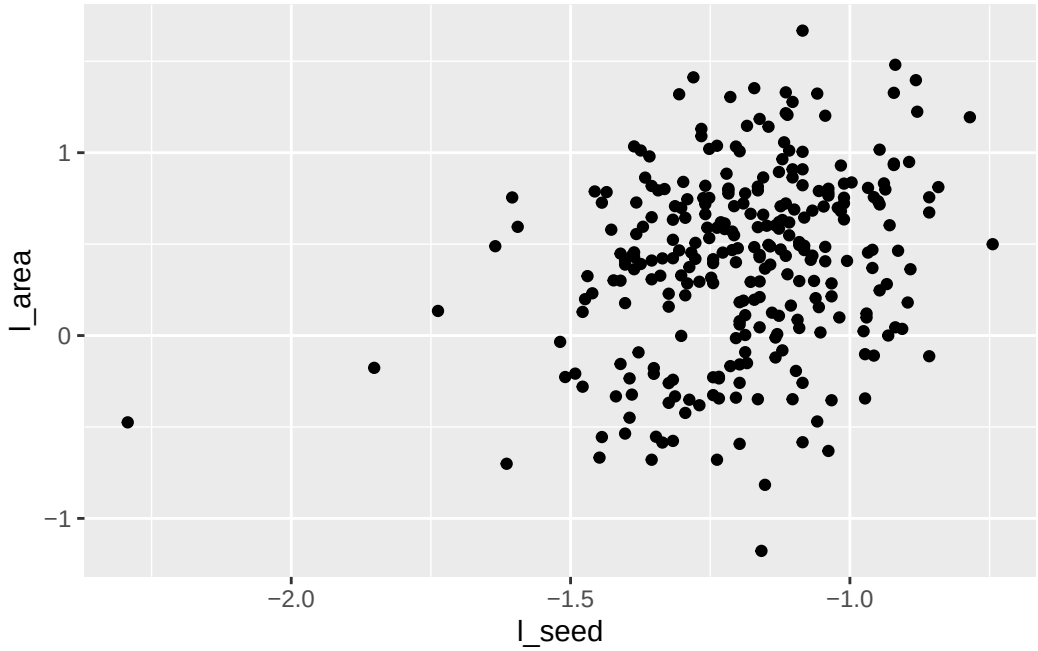
```
  tray block ind seed_area
1 lc_1     1   1     0.322
2 lc_1     1   2     0.287
3 lc_1     1   3     0.286
4 lc_1     1   4     0.274
5 lc_1     1   5     0.318
6 lc_1     1   6     0.239
```

```
d <- d %>%
  left_join(seed, by = join_by(tray, block, ind))
head(d)
```

```
  tray block ind   area age cross mother father id seed_area
1 lc_1     1   1  0.989   1    f2  B12_1 B12_11  1     0.322
2 lc_1     1   1  4.504   4    f2  B12_1 B12_11  1     0.322
3 lc_1     1   1  6.807   6    f2  B12_1 B12_11  1     0.322
4 lc_1     1   1  8.934   8    f2  B12_1 B12_11  1     0.322
5 lc_1     1   1 14.791  11    f2  B12_1 B12_11  1     0.322
6 lc_1     1   1 18.324  12    f2  B12_1 B12_11  1     0.322
```

I suspect that the log(seed_size) is linearly related to log(size0). Just to take a quick peak to see if that is more or less true, let's plot the log of size at the earliest time point of each individual against the log of seed size.

```
d %>%
  group_by(tray, block, ind) %>%
  filter(age == 1) %>%
  mutate(l_area = log(area),
         l_seed = log(seed_area)) %>%
  ggplot(aes(x = l_seed, y = l_area)) +
  geom_point()
```



Seems to be more or less linear, and likely closer at time point 0 so a linear model of l_size0 as a function of l_seed_size is probably a decent assumption.

The model will be very similar to model 4, but now it will have a the log of the initial size will be modeled: $\mu_{s0} = \alpha_{s0} + \beta_{s0} \times \log(seed)$.

$$
\begin{aligned}
\log(s) &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \log(s_0) + rgr \times age \\
\sigma &\sim \text{Exponential}(1) \\
\begin{bmatrix} \log(s_0) \\ rgr \end{bmatrix} &\sim \text{MVNormal}\left(\begin{bmatrix} \mu_{s0} \\ \mu_{rgr} \end{bmatrix}, \text{diag}(\tau)\mathbf{R}\text{diag}(\tau) \right) \\
\mu_{s0} &= \alpha_{s0} + \beta_{s0} \times \log(seed) \\
\alpha_{s0} &\sim \text{Normal}(0, 1) \\
\beta_{s0} &\sim \text{Normal}(0, 1) \\
\mu_{rgr} &\sim \text{Normal}(0.24, 0.1) \\
\tau &\sim \text{Exponential}(1) \\
\mathbf{R} &\sim \text{LKJcorr}(2)
\end{aligned}
\tag{5}
$$

Here is the stan program for this model:

```
data{
  int N; // number of observations
  int J; // number of individuals
  vector[N] l_size; // size observations
  vector[N] age; // age of observations
  array[N] int id; // index of individuals
  vector[J] l_seed; // seed sizes
  array[J] int id_seed; // id of individuals for seeds
}
parameters{
  real<lower=0> sigma; // standard deviation of errors
  real mu_rgr; // mean relative growth rate
  array[J] vector[2] Mu; // array of vectors carrying l_size0 and rgr
  vector<lower=0>[2] tau; // standard deviation of l_size0 and rgr
  corr_matrix[2] R; // correlation matrix between rgr and l_size0 error
  real alpha_s0; // average initial log size
  real beta_s0; // slope of log(size0) on log(seed size)
}
model{
  // priors
  sigma ~ exponential(1);
  mu_rgr ~ normal(0.24, 0.1);
  alpha_s0 ~ normal(0, 1);
  beta_s0 ~ normal(0, 1);
  tau ~ exponential(1);
  R ~ lkj_corr(2);

  array[J] vector[2] mu_s0_rgr;
  for(i in 1:J){
    mu_s0_rgr[i,1] = alpha_s0 + beta_s0 * l_seed[id_seed[i]];
    mu_s0_rgr[i,2] = mu_rgr;
  }


  Mu ~ multi_normal(mu_s0_rgr, quad_form_diag(R, tau));

  // likelihood
  // Define the expected value for each observation
  vector[N] mu;
  for(i in 1:N){
    mu[i] = Mu[id[i], 1] + Mu[id[i], 2] * age[i];
  }
```

```
  l_size ~ normal(mu, sigma);
}
```

Now I sorta need a short data set with just the seed size and id for each individual, and then the long data set of seedling size observations.

```
short <- d %>%
  filter(!duplicated(paste0(tray, "_", block, "_", ind))) %>%
  select(seed_area, id)
```

Now compile and fit the model.

```
# compile the model and get the data ready
mod <- cmdstan_model(here("notebooks/model.stan"))

dat <- list(
  N = nrow(d),
  J = length(unique(d$id)),
  l_size = log(d$area),
  age = d$age,
  id = d$id,
  # add the short data in
  l_seed = log(short$seed_area),
  id_seed = short$id
)

fit <- mod$sample(
  data = dat,
  chains = 4,
  parallel_chains = 4,
  show_messages = F,
  show_exceptions = F
)

fit$diagnostic_summary()
```

```
$num_divergent
[1] 0 0 0 0

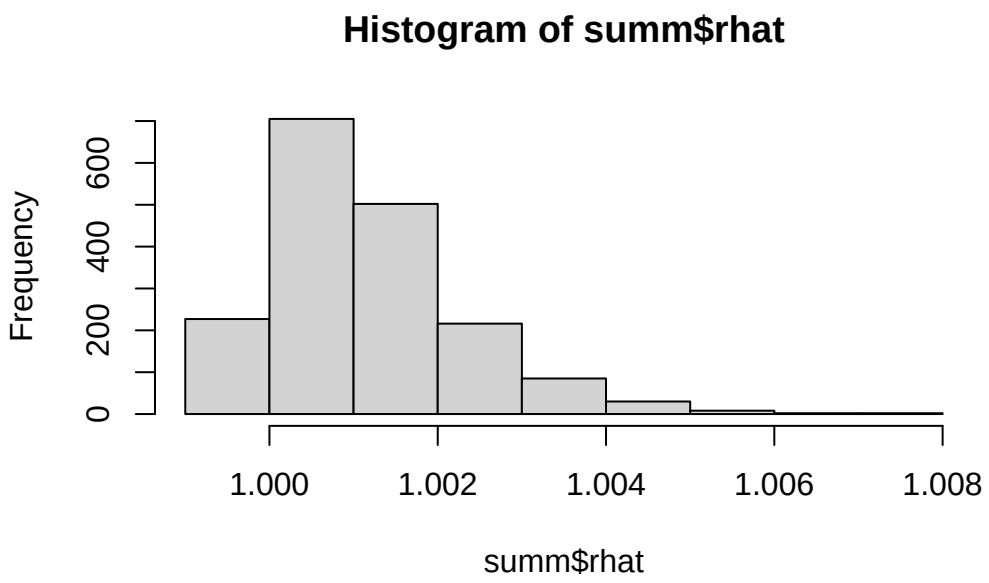
$num_max_treedepth
[1] 0 0 0 0
```

```
$ebfmi
[1] 0.8771490 0.8020284 0.7990594 0.9001202
```

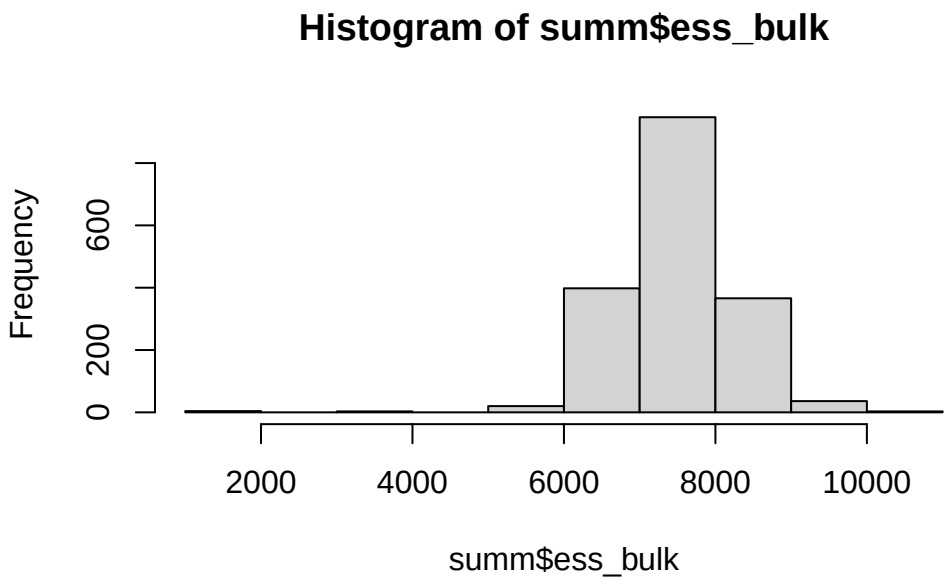
```
summ <- fit$summary()
```

It looks like, computationally it fit ok. Let's check the rhat and ess again.

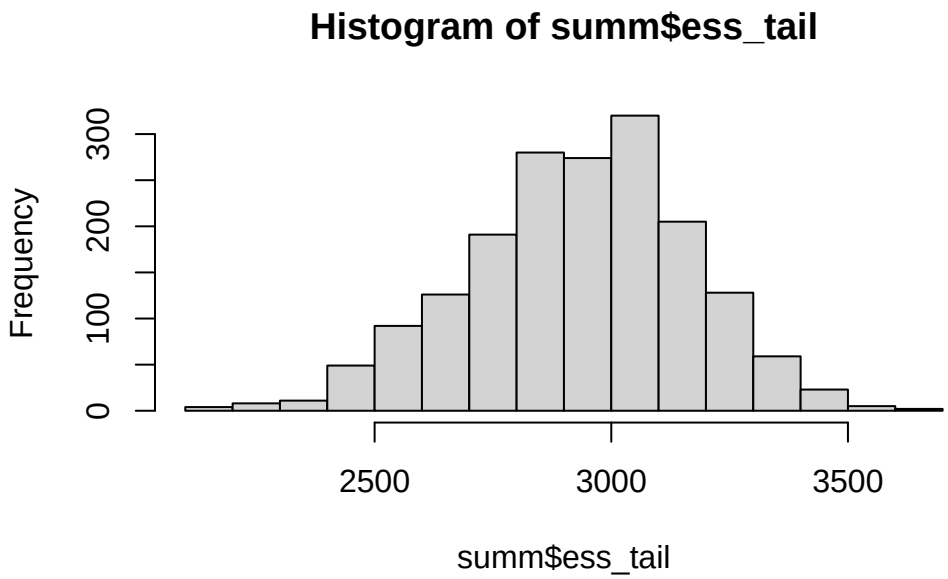
```
hist(summ$rhat)
```




```
hist(summ$ess_bulk)
```




```
hist(summ$ess_tail)
```



Still looking good. I should probably start evaluating model fit as well as computation. Let's add information about crosses first before doing that.